UNIDAD DE TRABAJO 2 - PROGRAMACIÓN ESTRUCTURADA

FICHA TÉCNICA DE LA UNIDAD

Unidad Trabajo	UT2. Programación estructurada
Resultados de Aprendizaje	RA2. Maneja los elementos necesarios en un lenguaje de programación, identificando los bloques fundamentales de construcción de un programa.
Criterios de Evaluación	CE2.a: Se ha identificado y operado adecuadamente con los diferentes tipos de datos en un programa. CE2.b: Se han escrito programas estructurados que recojan, procesen y generen información correctamente. CE2.c: Se han utilizado estructuras de control de flujo adecuadas en función del problema a resolver. CE2.d: Se han aplicado técnicas de depuración y pruebas para asegurar la corrección de los programas. CE2.e: Se han implementado buenas prácticas en la escritura de código, incluyendo documentación y uso de herramientas de control de versiones.
Tiempo	15 horas

ÍNDICE DE CONTENIDOS

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

2. FUNDAMENTOS DE PYTHON

2.1. El Concepto de Script frente a Programa Ejecutable

2.2. Variables, Ámbitos y Asignación Dinámica

2.3. Literales y Comentarios al Código

3. TIPOS DE DATOS PRIMITIVOS E IDENTIFICACIÓN

3.1. Datos Numéricos: Enteros (int) y Reales (float)

3.2. Operadores Aritméticos de Sistemas

3.3. Funciones Numéricas Predefinidas

3.4. Datos Booleanos y Operaciones Lógicas/Relacionales

3.5. Datos de Texto: Cadenas de Caracteres (strings)

4. ENTRADA Y SALIDA ESTÁNDAR

4.1. Captura de Datos con input() y Conversión de Tipos (Casteo)

4.2. Visualización con print() y Formateo Básico

5. ESTRUCTURAS DE CONTROL DE FLUJO LÓGICO

5.1. La Regla Sintáctica de la Indentación

5.2. Alternativa Condicional (if - elif - else)

5.3. Bucle Iterativo Condicional (while) y Controladores (break)

5.4. Bucle Iterativo (for) y Controladores (break/continue)

6. FUNCIONES. RECURSIVIDAD

6.1. Definición de Funciones

6.2. Ámbito de Variables: Variables Locales y Globales

6.3. Funciones Recursivas

6.4. Devolución de Múltiples Valores en Funciones

6.5. Identificación del Programa Principal (main) y Control de Ejecución

7. MÓDULOS

7.1. Definición e Importación de Módulos

7.2. Principales Módulos Estándar de Python

7.3. Creación y Uso de Librerías Propias (Módulos Personalizados)

7.4. Ejemplos Prácticos de Módulos Comunes de Propósito General y Sistemas

1. INTRODUCCIÓN Y CONTEXTUALIZACIÓN

El aprendizaje de la programación estructurada constituye el primer paso metodológico real para la cualificación de un Administrador de Sistemas hacia el ámbito de la automatización (*scripting*) y la filosofía DevOps. Esta metodología se basa en un principio fundamental: **las instrucciones de un script se ejecutan de manera lineal y secuencial**, una detrás de otra, salvo cuando alteramos deliberadamente este orden natural mediante el uso de decisiones condicionales o repeticiones de código (bucles).

Relación Curricular:

Esta unidad didáctica se fundamenta de forma directa sobre la lógica algorítmica y el pensamiento computacional adquiridos en la unidad anterior utilizando pseudocódigo en PSeInt (UT1). A nivel interdisciplinar, se conecta con:

- **Administración de Sistemas Operativos (2º ASIR):** Al sentar las bases sintácticas necesarias para sustituir los antiguos scripts en Bash por scripts avanzados de Python capaces de realizar comprobaciones básicas de uso de hardware y control de credenciales de usuario.
- **Gestión de Bases de Datos (1º ASIR):** Al capacitar al alumno en la comprensión del uso de operadores lógicos y relacionales, fundamentales para comprender el filtrado y estructuración de información en sistemas de bases de datos.

2. FUNDAMENTOS DE PYTHON

2.1. El Concepto de Script frente a Programa Ejecutable

A diferencia de los lenguajes compilados de bajo o alto nivel (como C o C++), que traducen todo su código a un archivo binario ejecutable antes de correr en la máquina, **Python es un lenguaje de alto nivel interpretado**.

Esto significa que las instrucciones se guardan en un archivo de texto plano con la extensión .py. Al ejecutarlo, un programa intermedio denominado **intérprete** lee y procesa el código línea a línea en tiempo de ejecución.

- **Ventajas:** No hay fase previa de compilación. Puedes escribir tu código e implementarlo de manera inmediata en servidores con distintos sistemas operativos (Linux, Windows o macOS) siempre que tengan instalado el intérprete de Python.

2.2. Variables, Ámbitos y Asignación Dinámica

Una **variable** es una referencia lógica o un nombre que apunta a un espacio en la memoria RAM del servidor para guardar un dato que puede cambiar durante la ejecución del script.

Para guardar un dato dentro de una variable, utilizamos el operador de asignación =. Python utiliza un mecanismo denominado **tipado dinámico**: no requiere que declares de qué tipo es la variable antes de usarla, sino que el propio intérprete detecta el tipo de dato de forma automática en el momento en que le asignas un valor.

- **Ámbito de una variable (Scope):** Hace referencia a las partes del programa donde una variable existe y se puede leer o escribir.
    - **Variables Globales:** Se definen en el flujo principal de nuestro script y son accesibles desde cualquier parte de nuestro código actual.
    - **Variables Locales:** Se definen dentro de una estructura específica (como una función o bloque de control) y solo existen mientras dure la ejecución de ese bloque.

2.3. Literales y Comentarios al Código

- **Literales:** Son valores fijos que se escriben directamente en el código del programa. Por ejemplo, en la expresión umbral = 90, el número 90 es un literal numérico entero.
- **Comentarios:** Son anotaciones en el código que el intérprete de Python ignora por completo durante la ejecución. En administración de sistemas, es obligatorio documentar qué hace cada script o línea compleja. Los comentarios de una sola línea se inician siempre con el carácter almohadilla #.

3. TIPOS DE DATOS PRIMITIVOS E IDENTIFICACIÓN

3.1. Datos Numéricos: Enteros (int) y Reales (float)

Python trabaja de manera nativa con dos tipos de datos numéricos principales:

- **Enteros (int):** Representan números sin parte decimal, ya sean positivos, negativos o el cero.
- **Reales o de punto flotante (float):** Representan números con coma o decimales (se escribe con punto .).

Podemos utilizar la función integrada `type()` para averiguar en cualquier momento la clase de dato que almacena una variable.

Conversión de tipos:

- `int(x)`: Convierte el valor a entero.
- `float(x)`: Convierte el valor a float.

A) Ejemplo Básico:

```
# Definición de dos tipos de números simples
edad_usuario = 25
altura_metros = 1.78
print(type(edad_usuario)) # Imprime: <class 'int'>
print(type(altura_metros)) # Imprime: <class 'float'>
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: inventario_servidor.py
# Guardamos valores para monitorizar el estado físico de un nodo
servidores_en_rack = 14 # Tipo 'int': Unidades enteras indivisibles
consumo_electrico_amperios = 4.85 # Tipo 'float': Medición precisa con decimales
print("Tipo de la variable servidores_en_rack:")
print(type(servidores_en_rack))
print("Tipo de la variable consumo_electrico_amperios:")
print(type(consumo_electrico_amperios))
```

3.2. Operadores Aritméticos de Sistemas

Símbolos matemáticos utilizados para manipular los valores numéricos:

Operador	Operación	Ejemplo	Resultado
+	Suma	5 + 2	7
-	Resta	5 - 2	3
*	Multiplicación	5 * 2	10
/	División (siempre devuelve decimal)	5 / 2	2.5

`//`	División entera (elimina los decimales)	5 // 2	2
`%`	Módulo (devuelve el resto de la división)	5 % 2	1
`**`	Potencia	5 ** 2	25

(Nota: Los operadores unarios + y - actúan para representar la polaridad positiva o negativa de un valor numérico).

A) Ejemplo Básico:

```
# Operaciones matemáticas directas con variables
dividendo = 13
divisor = 4
resultado_division_entera = dividendo // divisor
resto_sobrante = dividendo % divisor
print(resultado_division_entera) # Muestra: 3
print(resto_sobrante) # Muestra: 1
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: particionamiento_disco.py
# Propósito: Evaluar el espacio libre sobrante de una cabina tras realizar backups
capacidad_total_bloques = 175
bloques_usados_por_backup = 30
# Calcular cuántos backups exactos caben y cuánto espacio de bloques sobra
backups_posibles = capacidad_total_bloques // bloques_usados_por_backup
bloques_sobrantes = capacidad_total_bloques % bloques_usados_por_backup
print("backups que se pueden almacenar en la cabina:")
print(backups_posibles) # Muestra: 5
print("Bloques de almacenamiento que quedan vacíos al final:")
print(bloques_sobrantes) # Muestra: 25 (el resto de la división)
```

3.3. Funciones Numéricas Predefinidas

Python tiene funciones cargadas en su núcleo para manipular datos numéricos directamente:

- `abs(x)`: Devuelve el valor absoluto de un número (su magnitud sin signo).
- `pow(x, y)`: Eleva el número base x al exponente y.

- `round(x, [y])`: Redondea un número real x al número de decimales indicado en y.
- `bin(x)`: Convierte un número entero a su representación binaria.
- `hex(x)`: Convierte un número entero a su representación hexadecimal.
- `divmod(x, y)`: Devuelve la división entera y el módulo de manera conjunta.

A) Ejemplo Básico:

```
# Redondeo simple y valor absoluto
numero_negativo = -15.678
print(abs(numero_negativo)) # Muestra: 15.678
print(round(numero_negativo, 2)) # Muestra: -15.68
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: analisis_red.py
# Propósito: Convertir parámetros de red decimales a binario/hex para análisis manual
primer_octeto_ip = 192
latencia_ping_ms = 45.728
print("Octeto IP convertido a Binario:")
print(bin(primer_octeto_ip)) # Muestra: '0b11000000'
print("Octeto IP convertido a Hexadecimal:")
print(hex(primer_octeto_ip)) # Muestra: '0xc0'
print("Latencia media de la red redondeada a un único decimal:")
print(round(latencia_ping_ms, 1)) # Muestra: 45.7
```

Quizás eches en falta más operaciones que podemos realizar sobre los números. En el módulo `math` encontramos muchas de estas operaciones. Para utilizarlas vamos a importar el módulo, por ejemplo para realizar una raíz cuadrada:

```
import math
math.sqrt(9)
```

3.4. Datos Booleanos y Operaciones Lógicas/Relacionales

Un **booleano** (`bool`) es un tipo de dato que solo puede tomar uno de dos estados posibles: `True` (Verdadero) o `False` (Falso). Son fundamentales para definir condiciones de

seguridad y saltos de flujo lógico.

- **Operadores Relacionales:** `==` (Igual a), `!=` (Distinto de), `<`, `>`, `<=`, `>=`.
- **Operadores Lógicos:** `and` (Y lógico), `or` (O lógico), `not` (Negación).

Cuando se evalúa una expresión, hay determinados valores que se interpretan como `False`: `False`, cualquier número 0 (`0`, `0.0`), cualquier secuencia vacía (`[]`, `()`, `''`) y cualquier diccionario vacío (`{}`).

A) Ejemplo Básico:

```
# Comparación lógica de variables numéricas
limite_edad = 18
edad_usuario = 16
puede_entrar = edad_usuario >= limite_edad
print(puede_entrar) # Muestra: False
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: auditoria_firewall.py
# Comprobamos si un puerto solicitado por una aplicación es seguro
puerto_solicitado = 80
puerto_ssh = 22
puerto_https = 443
```

3.5. Datos de Texto: Cadenas de Caracteres (strings)

Las cadenas de texto se identifican por ir encerradas entre comillas simples (`'`) o dobles (`"`).

- **Operador + (Concatenación):** Permite unir dos o más cadenas de texto para formar una sola.
- **Operador * (Repetición):** Permite repetir una cadena un número determinado de veces.

A) Ejemplo Básico:

```
# Concatenación simple de palabras
saludo = "Hola"
```

```
destino = "Mundo"
mensaje_completo = saludo + " " + destino
print(mensaje_completo) # Muestra: Hola Mundo
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: constructor_rutas.py
# Propósito: Generar automáticamente la ruta de volcado de logs para una máquina virtual
directorio_base_linux = "/var/log/nginx"
nombre_archivo_log = "error.log"
# Generación automática de la ruta completa usando concatenación
ruta_completa_sistema = directorio_base_linux + "/" + nombre_archivo_log
print("El archivo de registro se guardará en:")
print(ruta_completa_sistema) # Muestra: /var/log/nginx/error.log
```

Indexación: Se puede obtener el dato de una secuencia indicando su índice. En este caso es posible obtener un carácter de la cadena indicando su posición (empezando por el índice 0); por ejemplo: `cadena[0]`.

Para obtener la longitud de una cadena (número de caracteres que contiene), utilizamos la función `len()`.

Las cadenas se comparan carácter a carácter: en el momento en que dos caracteres no son iguales se comparan alfabéticamente (se evalúa según su código Unicode).

4. ENTRADA Y SALIDA ESTÁNDAR

4.1. Captura de Datos con input() y Conversión de Tipos (Casteo)

La función `input(mensaje)` detiene la ejecución del programa y espera a que el operador escriba una entrada por teclado y pulse Intro.

REGLA DE ORO DE INPUT: La función `input()` siempre devuelve la información capturada en forma de texto (tipo `str`).

Si necesitas realizar operaciones numéricas, debes transformarlo explícitamente utilizando funciones de conversión de tipos (también conocidas como funciones de casteo):

- `int(variable)`: Convierte un texto numérico compatible a entero (ej. "22" a 22).

- `float(variable)`: Convierte un texto numérico compatible a decimal (ej. "12.5" a 12.5).
- `str(variable)`: Convierte un valor numérico a texto.

A) Ejemplo Básico:

```
# Lectura de un número y cálculo de su cuadrado
numero_texto = input("Introduce un número entero: ")
numero_entero = int(numero_texto) # Casteo a entero obligatorio
resultado_cuadrado = numero_entero ** 2
print(resultado_cuadrado)
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: aprovisionamiento_cuotas.py
# Propósito: Calcular el espacio total en disco requerido para dar de alta varios usuarios
cuota_por_usuario_gb = 15
usuarios_nuevos_texto = input("Introduce la cantidad de usuarios que se darán de alta en el
usuarios_nuevos = int(usuarios_nuevos_texto) # Casteo a entero para poder multiplicar
espacio_total_necesario_gb = usuarios_nuevos * cuota_por_usuario_gb
print("Espacio total aproximado que se debe reservar en la partición (GB):")
print(espacio_total_necesario_gb)
```

4.2. Visualización con print() y Formateo Básico

La función `print()` envía información al canal de salida estándar (la terminal). Podemos formatear o estructurar la salida mediante comas (`,`) o concatenando con el signo `+` (lo que requiere usar `str()` en las partes numéricas).

A) Ejemplo Básico:

```
# Diferentes formas de mostrar variables juntas
anio = 2026
print("El año actual es", anio)
print("El año actual es " + str(anio))
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: telemetria_servidores.py
temperatura_cpd_grados = 21.5
cantidad_servidores_activos = 8
# Formateo de salida combinando cadenas y valores de monitorización
print("Estado del CPD: Servidores encendidos:",
      cantidad_servidores_activos, "| Temperatura del CPD:",
      temperatura_cpd_grados, "grados Celsius.")
```

Con la función `print()` podemos indicar el formato con el que se van a mostrar los datos, por ejemplo:

```
>>> print("El producto %s cantidad=%d precio=%.2f" % ("cesta", 23, 13.456))
El producto cesta cantidad=23 precio=13.46
```

5. ESTRUCTURAS DE CONTROL DE FLUJO LÓGICO

5.1. La Regla Sintáctica de la Indentación

A diferencia de otros lenguajes de programación como C, Java o PHP (que delimitan los bloques internos de control mediante el uso de llaves `{ }`), **Python utiliza la indentación (el sangrado o espacios en blanco)** como un requisito de sintaxis obligatorio.

Todas las instrucciones de un bloque de código común deben estar perfectamente alineadas a la derecha con el mismo número de espacios (la norma internacional de estilo fija de forma preferente **4 espacios en blanco**).

```
if condicion_analizada:
    instruccion_dentro_del_bloque_1
    instruccion_dentro_del_bloque_1
instruccion_fuera_del_bloque # Al volver al margen inicial, termina el bloque anterior
```

5.2. Alternativa Condicional (if - elif - else)

Permite desviar la ejecución secuencial del script. Evalúa condiciones booleanas de forma jerárquica y secuencial. En el momento en que una condición es verdadera, ejecuta el bloque

de instrucciones indentado correspondiente y salta al final de la estructura de control condicional.

A) Ejemplo Básico:

```
# Determinación simple de rangos numéricos
nota = int(input("Introduce tu nota del examen (1-10): "))
if nota >= 5:
    print("Has aprobado el examen.")
else:
    print("Has suspendido el examen.")
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: analizador_http.py
# Propósito: Evaluar el comportamiento de un servidor web según el código de respuesta devue
codigo_estado_http = int(input("Introduce el código de estado HTTP recibido del servidor wel
if codigo_estado_http == 200:
    print("Diagnóstico: El servidor web responde correctamente (OK).")
elif codigo_estado_http == 301 or codigo_estado_http == 302:
    print("Diagnóstico: El recurso solicitado está configurado como una Redirección.")
elif codigo_estado_http == 404:
    print("Diagnóstico: Error del cliente. Recurso no encontrado en el servidor.")
elif codigo_estado_http == 500:
    print("Diagnóstico: Error crítico interno del servidor web.")
else:
    print("Diagnóstico: Código de respuesta HTTP no reconocido o alternativo.")
```

5.3. Bucle Iterativo Condicional (while) y Controladores (break)

El bucle `while` evalúa una condición booleana y, **mientras esta condición se mantenga verdadera (True)**, repite indefinidamente la ejecución del bloque de código indentado a su derecha.

- **Instrucción break:** Permite interrumpir de forma abrupta e inmediata la ejecución del bucle, saltando directamente a la primera línea de código que se encuentre fuera del bloque repetitivo.

- **Instrucción continue:** Deja de ejecutar las restantes instrucciones del bucle y vuelve a iterar.

A) Ejemplo Básico:

```
# Un bucle que cuenta de forma progresiva del 1 al 5
contador = 1
while contador <= 5:
    print(contador)
    contador = contador + 1 # Incremento básico de la variable de control
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: shell_segura.py
# Propósito: Simular la solicitud persistente de una credencial para acceder a la terminal c
contrasena_correcta = "smr2023"
intentos_restantes = 3
print("--- PANEL DE SEGURIDAD DE ACCESO SSH ---")
while intentos_restantes > 0:
    clave_usuario = input("Introduce la contraseña de administración del switch: ")
    if clave_usuario == contrasena_correcta:
        print("Acceso concedido. Cargando sesión shell...")
        break # Interrumpe y sale del bucle 'while' inmediatamente al tener éxito
    else:
        intentos_restantes = intentos_restantes - 1
        print("Contraseña incorrecta.")
        print("Intentos de acceso restantes:", intentos_restantes)
        if intentos_restantes == 0:
            print("SISTEMA BLOQUEADO: Se ha superado el número máximo de intentos permitidos
```

En pseudocódigo podemos encontrar una instrucción *Repetir - Hasta Que* que nos permite ejecutar una secuencia de instrucciones hasta que la condición sea verdadera. Con esta estructura la secuencia de instrucciones que forma el cuerpo del ciclo se ejecuta una vez y luego se evalúa la condición. Esta estructura de control la podemos encontrar en muchos lenguajes de programación, pero en Python no existe nativamente. Podemos simular su comportamiento utilizando un bucle con `while True` y la instrucción `break`:

```
secreto = "asdasd"
while True:
    clave = input("Dime la clave: ")
```

```
    if clave != secreto:
        print("Clave incorrecta!!!")
    if clave == secreto:
        break
print("Bienvenido!!!")
print("Programa terminado")
```

5.4. Bucle Iterativo (for) y Controladores (break/continue)

El bucle `for` se emplea cuando sabemos de antemano cuántas veces queremos repetir una acción o cuando queremos recorrer elementos de una secuencia, como una cadena, una lista o un rango numérico.

- **Instrucción break:** Interrumpe la ejecución del bucle y sale inmediatamente del bloque repetitivo.
- **Instrucción continue:** Aborta la iteración actual y pasa directamente a la siguiente.

A) Ejemplo Básico:

```
# Un bucle que recorre los valores del 1 al 5
for contador in range(1, 6):
    print(contador)
```

B) Ejemplo Práctico (Perfil ASIR):

```
# Script: auditoria_servicios.py
# Propósito: Revisar el estado de varios servicios de red y marcar los problemas
servicios = ["ssh", "apache2", "mysql", "docker"]
for servicio in servicios:
    print("Comprobando servicio:", servicio)
    if servicio == "mysql":
        print("Advertencia: el servicio mysql requiere revisión manual.")
        continue
    print("Estado OK: servicio disponible.")
```

La función `range()` es la forma más habitual de generar secuencias numéricas para un bucle `for`. Por ejemplo, `range(1, 6)` genera los valores 1, 2, 3, 4 y 5. Esto resulta especialmente

útil para recorrer listas, ejecutar acciones repetidas un número concreto de veces o automatizar tareas de monitorización en sistemas.

```
# Ejemplo de recorrido con números y pausa de control
for numero in range(1, 11):
    if numero % 2 == 0:
        continue
    print("Número impar detectado:", numero)
    if numero == 7:
        print("Se ha alcanzado el límite de revisión.")
        break
```

6. FUNCIONES. RECURSIVIDAD

6.1. Definición de Funciones

Veamos un ejemplo de definición de función:

```
def factorial(n):
    """Calcula el factorial de un número"""
    resultado = 1
    for i in range(1, n + 1):
        resultado *= i
    return resultado
```

Y para utilizar la función:

```
>>> factorial(5)
120
```

6.2. Ámbito de Variables: Variables Locales y Globales

El ámbito de una variable es el lugar en el código fuente donde esta variable se puede usar. Una variable local declarada dentro de una función no se puede utilizar fuera de esa función. Veamos un ejemplo:

```
>>> def operar(a, b):
...     suma = a + b
...     resta = a - b
...     print(suma, resta)
...
>>> operar(4, 5)
9 -1
>>> resta
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'resta' is not defined
```

Podemos definir variables globales, que serán visibles en todo el módulo. Se recomienda declararlas en mayúsculas por convención:

```
>>> PI = 3.1415
>>> def area(radio):
...     return PI * radio ** 2
...
>>> area(2)
12.566
```

Parámetros formales y reales

- **Parámetros formales:** Son las variables que recibe la función y se especifican al definirla. Su contenido se recibe al realizar la llamada a la función a partir de los parámetros reales. Los parámetros formales son variables locales dentro de la función.
- **Parámetros reales:** Son las expresiones o valores concretos que se pasan en la llamada de la función; sus valores se enlazan o copian en los parámetros formales.

Paso de parámetros por valor o por referencia

En Python el paso de parámetros es siempre por asignación de referencias de objetos. El lenguaje no trabaja con el concepto clásico de variables contenedoras, sino con **objetos y referencias**. Al realizar la asignación `a = 1` no se considera que *"a contiene el valor 1"*, sino que *"a referencia al objeto 1"*. De este modo, en comparación con otros lenguajes, suele considerarse que en Python los parámetros se pasan por referencia.

Evidentemente, si se pasa un valor de un **objeto inmutable** (como enteros, floats, cadenas o tuplas), su valor no se podrá alterar fuera de la función:

```
>>> def f(a):
...     a = 5
>>> a = 1
>>> f(a)
>>> a
1
```

Sin embargo, si pasamos un **objeto mutable** (como listas o diccionarios), sí podremos alterar su estado interno:

```
>>> def f(lista):
...     lista.append(5)
...
>>> l = [1, 2]
>>> f(l)
>>> l
[1, 2, 5]
```

Aunque podemos modificar el parámetro real cuando los objetos pasados son mutables, no suele ser recomendable hacerlo directamente como efecto secundario. En otros lenguajes es necesario porque no existe la opción de retornar múltiples valores, pero en Python podemos retornar cómodamente tuplas o listas mediante la instrucción `return`.

Devolución de información

Una función en Python puede devolver información utilizando la sentencia `return`. La instrucción `return` puede retornar cualquier tipo de resultado o estructura de datos (por ejemplo, múltiples valores empaquetados en una tupla, lista o diccionario). Tan pronto como el flujo llega a la sentencia `return`, entrega el resultado y finaliza la ejecución de la función.

Llamadas a una función

Cuando se llama a una función se deben indicar los parámetros reales necesarios. La llamada a una función se evalúa como una expresión cuyo valor y tipo corresponden al valor retornado. Si la función no incluye una instrucción `return` explícita, devolverá por defecto `None`.

```
>>> def cuadrado(n):
...     return n * n
...
>>> a = cuadrado(2)
>>> cuadrado(3) + 1
10
>>> cuadrado(cuadrado(4))
256
>>> type(cuadrado(2))
<class 'int'>
```

6.3. Funciones Recursivas

La versión iterativa de esta función se ha mostrado en el apartado 6.1; a continuación se implementa de forma recursiva.

El factorial de un número también puede ser calculado de forma recursiva: el factorial de 0 o de 1 es 1, y el factorial de cualquier otro número es igual a dicho número multiplicado por el factorial del número anterior (`n - 1`).

Una **función recursiva** es aquella que durante su ejecución realiza llamadas a sí misma. Por ello, siempre es imprescindible definir un **caso base** que detenga la cadena de llamadas sucesivas:

```
def factorial(num):
    if num == 0 or num == 1:
        return 1
    else:
        return num * factorial(num - 1)
```

Invocación de la función:

```
>>> factorial(6)
720
```

6.4. Devolución de Múltiples Valores en Funciones

A diferencia de la mayoría de los lenguajes de programación tradicionales, en Python una función puede retornar más de un valor simultáneamente dentro de la misma sentencia `return`.

Para ello, basta con separar los valores o variables devueltas mediante comas. Internamente, Python empaqueta automáticamente estos datos en una **tupla** inmutable. Al recibir el resultado en la llamada a la función, podemos desempaquetar cada valor de forma directa en variables independientes.

Ejemplo Práctico:

```
def calcular_operaciones(a, b):
    """Devuelve la suma, la resta y la multiplicación de dos números"""
    suma = a + b
    resta = a - b
    producto = a * b
    return suma, resta, producto  # Devuelve múltiples valores separados por comas

# Asignación por desempaquetamiento directo
res_suma, res_resta, res_mult = calcular_operaciones(10, 5)
print("Suma:", res_suma)          # Muestra: Suma: 15
print("Resta:", res_resta)        # Muestra: Resta: 5
print("Multiplicación:", res_mult) # Muestra: Multiplicación: 50

# Si capturamos el resultado en una sola variable, veremos que es una tupla
resultado_tupla = calcular_operaciones(10, 5)
print(resultado_tupla)            # Muestra: (15, 5, 50)
print(type(resultado_tupla))      # Muestra: <class 'tuple'>
```

6.5. Identificación del Programa Principal (main) y Control de Ejecución

Cuando un programa en Python comienza a crecer y definimos múltiples funciones auxiliares, surge una pregunta clave: **¿cómo distinguimos el código de las funciones de la parte principal del script encargada de iniciar la ejecución?**

A diferencia de lenguajes como C, Java o C#, donde es obligatorio declarar un método o función de entrada `main()`, en Python el intérprete lee el archivo secuencialmente de arriba abajo y ejecuta directamente cualquier instrucción situada en el nivel de indentación raíz (sin

sangría). Las sentencias `def` se limitan a cargar las funciones en memoria, pero no ejecutan su lógica interna hasta que son invocadas.

Para estructurar el código de forma limpia, modular y profesional, se aplican dos patrones estándar en la industria:

- **1. Empaquetar la lógica de inicio en una función `main()`:** En lugar de dejar variables e instrucciones sueltas al final del archivo, se agrupan en una función principal (denominada convencionalmente `def main():` o `def principal():`). En ella se solicitan los datos al usuario, se llama a las funciones secundarias y se presentan los resultados.
- **2. El modismo de control `if __name__ == "__main__":`:** Python asigna automáticamente a la variable especial interna `__name__` el valor literal `"__main__"` cuando el script es ejecutado directamente desde la terminal (por ejemplo: `python3 programa.py`). En cambio, si el archivo es importado desde otro script (`import programa`), `__name__` contendrá el nombre del propio archivo (`"programa"`).

Al utilizar la condición `if __name__ == "__main__":` para invocar a `main()`, garantizamos que el programa principal solo se ejecutará cuando el usuario lance ese fichero directamente, evitando que se ejecute de manera accidental si en el futuro otro script decide importar alguna de sus funciones.

Estructura canónica de un script en Python con funciones y `main()`:

```
# ========================================================================
# 1. DEFINICIÓN DE FUNCIONES AUXILIARES (LÓGICA REUTILIZABLE)
# ========================================================================
def calcular_iva(precio, porcentaje=21):
    """Calcula el importe del impuesto a partir de una base y un porcentaje"""
    return precio * (porcentaje / 100)

def formatear_moneda(cantidad):
    """Devuelve la cantidad formateada con dos decimales y el símbolo de euro"""
    return f"{cantidad:.2f} €"

# ========================================================================
# 2. FUNCIÓN PRINCIPAL (PUNTO DE ENTRADA DEL PROGRAMA)
# ========================================================================
def main():
    """Coordina el flujo de ejecución principal del script"""
```

```
    print("--- CALCULADORA DE FACTURACIÓN ---")
    base_imponible = float(input("Introduce el precio base del producto: "))
    # Invocación de las funciones auxiliares
    impuesto = calcular_iva(base_imponible)
    total = base_imponible + impuesto
    print("Base:", formatear_moneda(base_imponible))
    print("IVA aplicado:", formatear_moneda(impuesto))
    print("Total a pagar:", formatear_moneda(total))

# ====================================================================
# 3. IDENTIFICADOR DEL PROGRAMA PRINCIPAL
# ====================================================================
if __name__ == "__main__":
    # Solo se ejecuta si el script se ejecuta de forma directa
    main()
```

7. MÓDULOS

7.1. Definición e Importación de Módulos

Definición de Módulo: Cada uno de los ficheros con extensión `.py` que creamos constituye un módulo. Los elementos creados dentro de un módulo (como funciones, clases o constantes) pueden ser importados para reutilizarse en otros módulos. El nombre de importación coincide con el nombre base del archivo.

Python cuenta con una extensa colección de módulos preinstalados que conforman su **biblioteca estándar**. Para utilizarlos en nuestros programas debemos recurrir a la sentencia `import`.

Podemos importar un módulo completo y hacer referencia a sus funciones mediante la notación de punto. Por ejemplo, para calcular una raíz cuadrada con el módulo matemático `math`:

```
>>> import math
>>> math.sqrt(9)
3.0
```

Si solo necesitamos una función concreta, podemos importarla directamente con `from ... import ...`, eliminando la necesidad de prefijar el nombre del módulo en cada invocación:

```
>>> from math import sqrt
>>> sqrt(9)
3.0
```

7.2. Principales Módulos Estándar de Python

Módulos matemáticos

- **Módulo `math`:** Proporciona funciones y constantes matemáticas avanzadas para cálculos algebraicos y de geometría básica.
    - `math.sqrt(x)`: Devuelve la raíz cuadrada flotante de `x`.
    - `math.pow(x, y)`: Eleva `x` a la potencia `y` (devuelve `float`).
    - `math.ceil(x)`: Redondea hacia arriba al entero más cercano (techo).
    - `math.floor(x)`: Redondea hacia abajo al entero más cercano (suelo).
    - `math.pi` / `math.e`: Constantes matemáticas de uso común (π y número e).

```
import math

# Ejemplo general: Cálculo de área y redondeos de un producto
radio = 5
area_circulo = math.pi * math.pow(radio, 2)
print("Área exacta del círculo:", area_circulo)
print("Redondeo hacia arriba (ceil):", math.ceil(area_circulo))  # 79
print("Redondeo hacia abajo (floor):", math.floor(area_circulo)) # 78
```

- **Módulo `random`:** Generación de números aleatorios y selecciones al azar, útil para juegos, sorteos, simulación de datos o asignación aleatoria de tareas.
    - `random.randint(a, b)`: Devuelve un número entero aleatorio comprendido entre a y b (ambos inclusive).
    - `random.choice(secuencia)`: Selecciona un único elemento al azar de una lista o cadena.
    - `random.sample(secuencia, k)`: Selecciona k elementos aleatorios de una lista sin repetir.

- `random.shuffle(lista)`: Mezcla o desordena de forma aleatoria los elementos de una lista in situ.

```
import random

# Ejemplo general: Juego de dados y sorteo de alumnos
dado = random.randint(1, 6)
estudiantes = ["Ana", "Carlos", "Beatriz", "David", "Elena"]
ganador = random.choice(estudiantes)

print("Resultado del dado:", dado)
print("Estudiante seleccionado al azar:", ganador)
```

Módulos de hora y fecha

- **Módulo `time`:** Gestión del tiempo basado en marcas temporales (segundos desde la época POSIX) y control de pausas en programas.
  - `time.sleep(segundos)`: Pausa la ejecución del script durante los segundos indicados.
  - `time.time()`: Obtiene la marca de tiempo actual en segundos; muy útil para medir cuánto tarda en ejecutarse un trozo de código.

```
import time

# Ejemplo general: Medición del tiempo de ejecución de un proceso
inicio = time.time()
print("Procesando datos...")
time.sleep(1.5)  # Simula una tarea pesada pausando 1.5 segundos
fin = time.time()
print("Tiempo total transcurrido:", round(fin - inicio, 2), "segundos.")
```

- **Módulo `datetime`:** Manipulación amigable de fechas, horas y cálculos de intervalos entre días u horas.
  - `datetime.now()`: Devuelve un objeto con la fecha y hora exacta actual.
  - `strftime(formato)`: Convierte una fecha en una cadena de texto personalizada.
  - `timedelta(days, hours, ...)`: Permite sumar o restar días y horas a una fecha.

```
from datetime import datetime, timedelta

# Ejemplo general: Cálculo de fecha de vencimiento
hoy = datetime.now()
plazo_devolucion = timedelta(days=15)
fecha_limite = hoy + plazo_devolucion

print("Fecha actual:", hoy.strftime("%d/%m/%Y"))
print("Fecha límite de devolución (+15 días):", fecha_limite.strftime("%d/%m/%Y"))
```

Módulos de interacción con el sistema

- **Módulo `os` :** Permite realizar operaciones generales con el sistema de archivos (consultar carpetas, crear directorios, verificar la existencia de ficheros).
  - `os.getcwd()` : Obtiene la ruta del directorio de trabajo actual.
  - `os.listdir(ruta)` : Devuelve una lista con los nombres de archivos y carpetas contenidos en un directorio.
  - `os.path.exists(ruta)` : Comprueba si existe un archivo o directorio en la ruta indicada.
  - `os.mkdir(nombre)` : Crea una nueva carpeta en la ruta de trabajo.

```
import os

# Ejemplo general: Comprobación y creación automática de carpetas
carpeta_destino = "mis_documentos"
if not os.path.exists(carpeta_destino):
    os.mkdir(carpeta_destino)
    print("Carpeta creada correctamente:", carpeta_destino)
else:
    print("La carpeta ya existe. Contenido actual:", os.listdir(carpeta_destino))
```

Funciones e Instrucciones Incorporadas para Trabajar con Cadenas (Built-in)

Las cadenas de texto (`str`) se utilizan en cualquier tipo de programa para procesar nombres, textos introducidos por usuario, limpiar datos de entrada y dar formato a informes. A continuación se presentan las funciones e instrucciones nativas más utilizadas:

- **`len(cadena)`:** Función integrada que devuelve la cantidad total de caracteres del texto.
- **`cadena.upper()` / `cadena.lower()` / `cadena.title()`:** Transforman el texto completo a mayúsculas, minúsculas o formato título (primera letra de cada palabra en mayúscula).
- **`cadena.strip()`:** Limpia y elimina los espacios sobrantes, tabulaciones o saltos de línea al principio y final de una cadena.
- **`cadena.replace(buscar, reemplazar)`:** Sustituye una subcadena o carácter por otro dentro del texto.
- **`cadena.split(separador)`:** Divide un texto en una lista de palabras tomando un separador por defecto (espacio en blanco, coma, guion, etc.).
- **`separador.join(lista)`:** Junta los elementos de una lista de cadenas en un solo texto mediante un separador.
- **`cadena.find(subcadena)`:** Busca la posición (índice) donde comienza un texto. Devuelve `-1` si no lo encuentra.
- **`cadena.startswith(prefijo)` / `cadena.endswith(sufijo)`:** Evalúan si una cadena comienza o termina con determinado texto (devuelven `True` o `False`).

Ejemplo Práctico de Formateo y Normalización de Texto (Uso General):

```
# Entrada de datos sin formatear desde un formulario de usuario
entrada_usuario = "  juan carlos pérez  "

# 1. Limpieza de espacios en los extremos y formateo de nombre propio
nombre_limpio = entrada_usuario.strip().title()
print("Nombre formateado:", nombre_limpio) # Muestra: Juan Carlos Pérez

# 2. División de palabras para separar nombre y apellidos
partes_nombre = nombre_limpio.split(" ")
print("Primer nombre:", partes_nombre[0])   # Muestra: Juan
print("Primer apellido:", partes_nombre[1]) # Muestra: Carlos

# 3. Sustitución de caracteres
frase = "Aprender Java es divertido"
frase_modificada = frase.replace("Java", "Python")
print(frase_modificada) # Muestra: Aprender Python es divertido

# 4. Comprobación de extensiones de archivos
```

```
archivo = "documento_presentacion.pdf"
if archivo.endswith(".pdf"):
    print("El archivo es un documento PDF válido.")
```

7.3. Creación y Uso de Librerías Propias (Módulos Personalizados)

Crear nuestra propia librería de funciones en Python es tan sencillo como agrupar funciones reutilizables dentro de un fichero con extensión `.py` y guardarlo en el mismo directorio de trabajo desde donde vayamos a ejecutar nuestro programa principal.

¿En qué carpeta debe estar la librería? Rutas de búsqueda y localización:
Para que la sentencia `import mi_libreria` funcione correctamente, el intérprete de Python debe saber en qué directorio buscar el archivo. Por defecto, Python busca de forma prioritaria en la **misma carpeta donde reside el script principal que estamos ejecutando** (es la primera entrada registrada en su lista de rutas `sys.path`). Si organizamos nuestras librerías en una subcarpeta (por ejemplo, dentro de una carpeta llamada `modulos/` o `librerias/`), debemos importarla indicando la ruta mediante la notación de punto: `import modulos.mi_libreria` o `from modulos import mi_libreria`. Si la librería se encontrase en una carpeta externa completamente separada del proyecto, Python no la encontrará y devolverá un error `ModuleNotFoundError`, a menos que dicha carpeta se agregue a la variable de entorno `PYTHONPATH` del sistema operativo o se añada dinámicamente en tiempo de ejecución mediante `sys.path.append("/ruta/a/la/carpeta")`.

Paso 1: Crear la librería propia con funciones generales (`mi_libreria.py`)

```
# Archivo: mi_libreria.py
# Librería reutilizable de utilidades generales

def es_monto_valido(cantidad):
    """Comprueba que una cantidad monetaria o valor sea positivo"""
    return cantidad > 0

def calcular_precio_final(precio_base, porcentaje_descuento=0):
    """Calcula el precio final aplicando un descuento opcional"""
    descuento = precio_base * (porcentaje_descuento / 100)
    return precio_base - descuento

def formatear_titulo(texto):
    """Genera una cabecera vistosa con bordes para la consola"""
```

```
    linea = "=" * (len(texto) + 6)
    print(linea)
    print("  ", texto)
    print(linea)

# ====================================================================
# BLOQUE DE PRUEBAS UNITARIAS (OPCIONAL)
# Gracias a if __name__ == "__main__": este código solo corre al ejecutar
# directamente "python3 mi_libreria.py". Si otro script hace "import mi_libreria",
# este bloque se ignorará por completo.
# ====================================================================
if __name__ == "__main__":
    print("Ejecutando pruebas locales de la librería:")
    formatear_titulo("TEST DE LIBRERÍA")
    print("¿100 es válido?:", es_monto_valido(100))
    print("Precio con descuento:", calcular_precio_final(100, 20))
```

Paso 2: Importar y reutilizar la librería en el script principal (`main.py`)

```
# Archivo: main.py
import mi_libreria

# Uso de las funciones generales importadas
mi_libreria.formatear_titulo("SISTEMA DE GESTIÓN DE TIENDA")

precio_producto = 150.0
descuento_aplicado = 15

if mi_libreria.es_monto_valido(precio_producto):
    total = mi_libreria.calcular_precio_final(precio_producto, descuento_aplicado)
    print("Precio base:", precio_producto, "€")
    print("Precio final con descuento:", total, "€")
```

7.4. Ejemplos Prácticos de Módulos Comunes de Propósito General y Sistemas

A continuación se detallan las librerías estándar de Python más utilizadas tanto en la resolución de tareas generales de programación como en la administración de redes y sistemas:

- **Módulo `sys` (Información del Intérprete y Parámetros de Entrada):**

Permite capturar argumentos pasados al programa desde la terminal de comandos (`sys.argv`) o detener limpiamente la ejecución del script (`sys.exit()`).

```
import sys

# Captura de argumentos recibidos por consola: python3 main.py Juan 25
print("Nombre del programa:", sys.argv[0])

if len(sys.argv) < 2:
    print("Error: Debes proporcionar al menos un argumento (tu nombre).")
    sys.exit(1) # Finaliza el programa indicando error

nombre_usuario = sys.argv[1]
print("Bienvenido al programa,", nombre_usuario)
```

- **Módulo `math` / `statistics` (Cálculos Estadísticos y Matemáticos):**

  Ideal para realizar análisis de datos sencillos como calcular medias, medianas y desviaciones sin necesidad de librerías externas.

```
import statistics

# Análisis de calificaciones o datos numéricos
notas = [7.5, 9.0, 6.0, 8.5, 10.0]
media = statistics.mean(notas)
mediana = statistics.median(notas)

print("Nota media de la clase:", media)     # Muestra: 8.2
print("Mediana de las notas:", mediana)     # Muestra: 8.5
```

- **Módulo `json` (Intercambio de Datos y Ficheros de Configuración):**

  Utilizado para guardar información de programas en formato JSON o consumir datos procedentes de APIs web públicas o bases de datos.

```
import json

# Convertir un diccionario de Python a formato de texto JSON
datos_usuario = {
    "nombre": "Laura",
    "edad": 28,
```

```
    "lenguajes": ["Python", "SQL", "HTML"]
}

# Generar cadena de texto JSON
json_resultado = json.dumps(datos_usuario, indent=4)
print("Representación en formato JSON:")
print(json_resultado)
```

- **Módulo `platform` (Información del Entorno de Ejecución):**

  Permite conocer en qué sistema operativo y arquitectura se está ejecutando el programa para adaptar su comportamiento.

```
import platform

print("Sistema Operativo:", platform.system())
print("Versión del Sistema:", platform.release())
print("Arquitectura de la CPU:", platform.machine())
print("Versión de Python utilizada:", platform.python_version())
```

- **Módulo `subprocess` (Ejecución de Comandos Externos):**

  Permite ejecutar programas externos o comandos de la consola del sistema operativo directamente desde Python y capturar su resultado.

```
import subprocess

# Ejecutar un comando externo según el sistema operativo
resultado = subprocess.run(["ping", "-c", "1", "127.0.0.1"], capture_output=True, text=

if resultado.returncode == 0:
    print("La prueba de red ha respondido con éxito.")
else:
    print("Error al ejecutar la comprobación de red.")
```